

NEXUS OS — Boiler Intelligence Platform

Real-time industrial boiler monitoring with physics simulation, ML anomaly detection, and a live LLM analyst that explains what the plant is doing in plain operator language.

Description

NEXUS OS is a decoupled, event-driven demo of an industrial boiler intelligence stack. A physics simulation engine models a fire-tube boiler and publishes ~1 Hz telemetry to an MQTT broker; an Isolation Forest anomaly detector scores that stream in real time; and an AI analyst turns those scores into operator-facing diagnosis cards, "Ask the Plant" chat answers, what-if simulations, and end-of-shift reports. Every layer communicates only over MQTT topics, so each one can be run, restarted, or replaced independently. A live dashboard (either the single-file `index.html` or the Next.js app in `frontend/`) subscribes over WebSocket and renders gauges, trend charts, a predictive risk panel, and the AI chat. It is built to run entirely on your machine — the broker, engines, and (optionally) a local Ollama model — with a hosted Groq API as an alternative LLM backend.

Getting Started

Dependencies

- **OS:** macOS or Linux (Windows via WSL2). Verified on macOS (Darwin) and Linux.
- **Docker + Docker Compose** — the simplest way to run the full stack.
- For running the pieces manually instead of via Docker:
 - **Python 3.10+** with the packages in `requirements.txt` (NumPy, scikit-learn, paho-mqtt, FastAPI, etc.)
 - **Node.js 18+** and npm — only for the `frontend/` Next.js dashboard (Next 16).
 - **Mosquitto** MQTT broker with WebSocket support (ports **1883** TCP + **9001** WebSocket).
- **An LLM backend** for the AI analyst — either:
 - a **GROQ_API_KEY** for hosted inference, or
 - a local **Ollama** install with a pulled model (e.g. `qwen3.5:4b`).
- Optional: `.env` file (copy from `.env.example`) for `GROQ_API_KEY`, `OLLAMA_BASE_URL`, `OLLAMA_MODEL`, and RAG/Qdrant settings.

Installing

Clone the repository and enter it:

```
git clone <repo-url> nexus-os-demo
cd nexus-os-demo
```

Create your environment file from the template and fill in the values you need:

```
cp .env.example .env
# edit .env – set GROQ_API_KEY, or OLLAMA_BASE_URL / OLLAMA_MODEL for local
inference
```

For the manual (no-Docker) path, install the Python and frontend dependencies:

```
pip install -r requirements.txt      # engine + AI analyst deps
cd frontend && npm install && cd ..  # dashboard deps (only if using the
Next.js UI)
```

No further file changes are required — the MQTT topic hierarchy, ports, and retention defaults are all preconfigured.

Executing program

Option A — Docker Compose (full stack, recommended)

Brings up the broker, physics engine, anomaly detector, forecasting engine, historian, RAG server, AI analyst, and the frontend together:

```
docker compose up --build
```

- Dashboard: `http://localhost:3000`
- MQTT broker: `1883` (TCP) / `9001` (WebSocket)
- RAG server: `8001`

Option B — Run the pieces manually

Start Mosquitto with WebSocket support on port 9001, then launch each service in its own terminal:

```
# Terminal 1 – physics engine
python engine/boiler_engine.py

# Terminal 2 – anomaly detector
python engine/anomaly_detector.py

# Terminal 3 – AI analyst
export GROQ_API_KEY="your-key-here"      # or configure Ollama in .env
python engine/ai_analyst.py

# Optional Terminal 4 – local historian (90+ days of telemetry in SQLite)
python engine/historian_service.py
```

Then open the dashboard, either:

```
# the single-file dashboard – just open it in a browser
open index.html

# or the Next.js dashboard
cd frontend && npm run dev                # http://localhost:3000
```

Driving the simulation

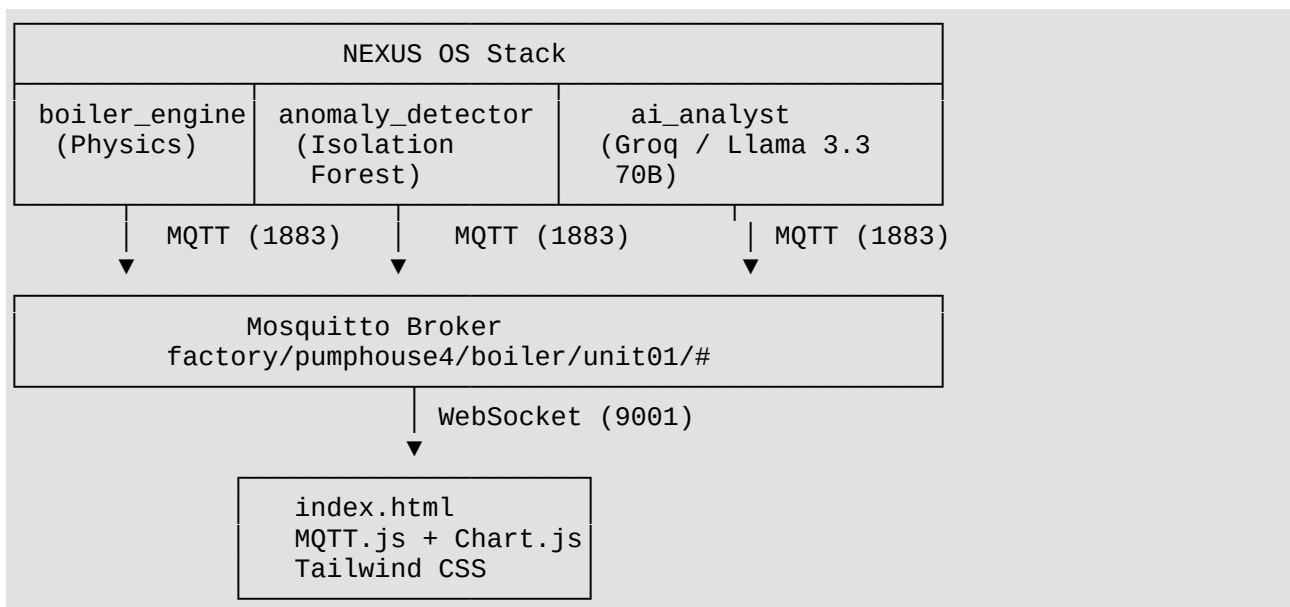
Switch scenarios live from the physics-engine terminal (Terminal 1) with single keypresses:

- `i` — ideal (clean reference run)
- `n` — normal operation

- d — degrading (tube-scaling fault)
- c — critical (drum level toward dry-fire)
- f — fault (flame failure / ESD)
- s — stop / back to normal
- r — reset

Then use the dashboard's AI chat to ask questions ("why is efficiency down?", "what if drum level drops to 180mm?"), or watch incident cards and the end-of-shift report appear as the plant state changes.

What's Inside



Features

1. Physics-Based Simulation Engine

`engine/boiler_engine.py` runs a state machine that models the thermodynamic behavior of an industrial fire-tube boiler. It's not random numbers — sensor relationships follow actual physics:

- **Steam temperature** is derived from pressure via a simplified Antoine equation approximation ($T_{sat} \approx 42.677 \times P^{0.2876} \times 10$), then adds 5 °C superheat
- **Boiler efficiency** is a function of stack heat loss (flue gas temp), excess air (O2%), and tube scaling degradation
- **Sensor lag** is modeled using exponential smoothing buffers — pressure has a 5-tap buffer, flue gas a 12-tap, replicating the sluggish response of real thermocouple and pressure transmitter installations

- **Gaussian noise** is applied on output, scaled per-sensor by realistic sigma percentages

Five operating modes that you can switch live from the terminal:

Mode	Key	What Happens
IDEAL	i	Clean reference run: no faults, no degradation, neutral environment, stable load
NORMAL	n	Controlled operation, all setpoints met
DEGRADING	d	Tube scaling fault — fuel flow climbs, flue gas temp rises, efficiency drops linearly
CRITICAL	c	Feedwater control struggles, drum level drops toward dry-fire threshold
FAULT	f	Flame failure / ESD — combustion stops, O2 hits atmospheric 20.9%

Data is published to a **Unified Namespace** topic hierarchy at 1 Hz:

```
factory/pumphouse4/boiler/unit01/
├── steam/pressure
├── steam/temperature
├── steam/flow
├── water/drum_level
├── water/feedwater_flow
├── combustion/fuel_flow
├── combustion/o2_percent
├── combustion/flue_gas_temp
├── safety/flame_status
├── safety/safety_valve
├── kpi/efficiency
├── kpi/heat_rate
├── system/heartbeat    ← full payload snapshot
├── system/mode
└── alerts
```

2. ML Anomaly Detector

`engine/anomaly_detector.py` runs **scikit-learn's Isolation Forest** (100 estimators, 5% contamination) directly on the live MQTT stream.

- Subscribes to the `heartbeat` topic and extracts 6 features: `steam_pressure`, `steam_temperature`, `drum_level`, `fuel_flow`, `flue_gas_temp`, `efficiency`
- Collects 40 samples during a **warm-up period** to establish a baseline before the model trains itself

- After warm-up, runs inference on every incoming reading
- Publishes a 0–100% anomaly score to `.../ai/anomaly_score` — higher means more anomalous

The score is inverted and scaled from the raw Isolation Forest decision function output:

```
anomaly_pct = clamp(0, 100, (1 - decision_score) × 50)
```

3. AI Analyst — Event-Driven LLM Layer

`engine/ai_analyst.py` sits downstream of the anomaly detector. It never runs the ML model — its job is to turn scores into language. When a threshold is crossed, it fires a **Groq API** call to `llama-3.3-70b-versatile` and publishes the result back to MQTT.

Three capabilities:

Incident Diagnosis Cards

- Fires when anomaly score crosses threshold OR a CRITICAL/HIGH alert lands
- Injects the last 15 seconds of telemetry trend + current snapshot into the prompt
- Forces JSON output: `probable_cause`, `severity`, `explanation`, `recommended_action`, `deviated_sensors[]`
- Debounced to one diagnosis per 30-second window to avoid alert storms
- Cards appear in the chat panel as work orders (`WO-XXXXX created`)

"Ask the Plant" Chat

- Operator types a free-form question; it gets published to `.../ai/question`
- The analyst injects the last 60 seconds of telemetry as context into the LLM prompt
- Maintains a 3-turn conversation history (`deque(maxlen=6)`) so follow-up questions like "and what about the drum?" resolve correctly
- Response published to `.../ai/response` → rendered in the dashboard with a typewriter effect

End-of-Shift Report

- Triggered by a "shift_report" type message on the chat topic
- Pulls from a `ShiftStats` object that has been accumulating since service start: uptime %, anomaly event count, alert counts by severity, efficiency delta, operating modes seen
- LLM writes the narrative summary and recommended follow-ups on top of the hard stats
- Rendered as a structured card with a 4-stat grid (uptime, anomalies, alerts, efficiency delta)

The telemetry context is managed by a **ring buffer** (`deque(maxlen=120)`) — enough for 2 minutes of 1 Hz data. `get_context(last_n)` returns it as a compact timestamped string that fits cleanly into the LLM prompt without blowing token count.

4. Real-Time Dashboard

`index.html` is a single-file dashboard — no build step, no framework. It connects directly to Mosquitto over WebSocket and renders everything live.

Predictive Intelligence Panel

A composite risk score is computed client-side from the `degradation_factor` plus threshold breach penalties:

```
risk = degradation × 100
      + (drum_level < 280 → +15)
      + (steam_pressure > 13 → +20)
      + (tube_health < 70 → +10)
      + (flame_status == 0 → 100)
```

Visualised as a colored progress bar: green → amber → orange → red (pulsing at CRITICAL).

Six Charts (Chart.js)

Chart	Type	What It Shows
Steam Pressure	Doughnut gauge	Live pressure vs 16 bar scale, color-coded by threshold
Drum Level	Doughnut gauge	Water level vs 600 mm scale
O ₂ Combustion	Bullet bar	Live O2% with zoned background (optimal 2–4%)
System Performance Trends	Multi-line	Efficiency %, tube health %, heat rate over time
Thermal Coupling	Divergence line	Steam temp vs flue gas temp — gap widens on fouling
Degradation Scatter	Scatter	Fuel flow (x) vs steam output (y) — cluster drifts on degradation

All charts buffer 60 data points and run at `animation: false` for smooth 1 Hz updates without layout reflow.

Intelligence Stream

A fixed-height monospaced terminal log (JetBrains Mono) scrolling the last 30 MQTT messages with timestamp, color-coded by severity. Capped at 30 lines to keep DOM size flat.

Alert / Event Timeline

Horizontal scrolling timeline strip. Each incoming alert drops a colored dot on the line with a timestamp label below and a hover tooltip showing the exact tag value vs threshold. Auto-scrolls right on new events. Capped at 20 nodes.

AI Chat Panel

The chat panel has a CSS `@property` animated conic-gradient border (`border-spin 7s linear infinite`) with a slow ambient glow pulse — purely CSS, no JS animation loop. Inside:

- Quick-prompt chips for one-tap common queries (health check, efficiency, failure prediction, maintenance priorities, shift report)
- Thinking state cycles through 4 phases ("Reading last 60s of live telemetry...", "Correlating sensor deviations...", etc.) at 1.4s intervals
- Answers render with a typewriter effect at 3 chars/16 ms per paragraph
- Sensor values like `10.2 bar` or `87.5%` in AI responses are automatically highlighted amber via a regex pass

5. Advanced Intelligence Features

Degradation Rate Forecaster — the dashboard runs a least-squares regression over the last ~45 samples of tube health. When a downward trend is detected, it projects time-to-breach of the 70% inspection threshold and shows a live countdown clock ("at this rate, tube health hits 70% in ~3m 40s") with the degradation rate in %/min. Recomputed every heartbeat; the countdown ticks every second between updates.

Combustion Tuning Advisor — a live one-liner recommendation computed client-side from O_2 vs the 2–4% optimal band, e.g. *"Air flow ~10% above optimal. Trim air damper to reduce excess O_2 from 4.8% → 3.2% and recover ~1.5% efficiency."* Excess-air percentage uses the lambda approximation $\lambda = 20.9 / (20.9 - O_2)$; recoverable efficiency matches the engine's own loss model. Also warns on low- O_2 (CO risk) and goes to standby on flame failure.

What-If Simulator — type a hypothetical into the chat ("what if drum level drops to 180mm?") and the AI walks the physical consequence chain step-by-step from the *current* live state, citing real protection thresholds. Rendered as a card with risk badge, numbered consequence chain, and operator actions. Questions containing "what if" are auto-routed to a dedicated simulation prompt in `ai_analyst.py`.

Multi-Turn Incident Memory — `ai_analyst.py` keeps a session-scoped `IncidentMemory` of alert episodes (deduplicating the 1 Hz alarm ticks into 60-second episodes) and past diagnoses. The history is injected into every diagnosis and chat prompt, so the AI can correlate: *"This is the third flue gas temp spike this session — the pattern matches tube fouling buildup, not a one-off transient."* Correlations surface in incident cards as a purple "Pattern detected" ribbon via the `pattern_note` field.

Tech Stack

Layer	Technology
MQTT broker	Mosquitto (ports 1883 TCP + 9001 WebSocket)
Data transport	MQTT QoS 1/2, JSON payloads

Layer	Technology
Physics engine	Python + NumPy
ML anomaly detection	scikit-learn <code>IsolationForest</code>
LLM inference	Groq Cloud API (<code>llama-3.3-70b-versatile</code>)
Dashboard	Vanilla JS + MQTT.js + Chart.js + Tailwind CSS
Styling	CSS custom properties, <code>@property</code> , <code>conic-gradient</code>

Local Historian

`engine/historian_service.py` is the default no-Docker historian path. It subscribes to the boiler heartbeat, alerts, anomaly score, diagnosis, and control-action topics, then stores them in SQLite at `historian/nexus_historian.db`.

- Default retention is 92 days, configurable with `HISTORIAN_RETENTION_DAYS`.
- Database path is configurable with `HISTORIAN_DB_PATH`.
- The AI analyst uses `engine/historian_client.py` to answer historical chat questions through safe query functions, not free-form SQL.
- Docker Compose includes an optional `historian` service if you later want to run the whole stack containerized.